SMART INDIA HACKATHON 2026 · PS 26122

Start Guide

Setup, verification and API reference — Phases 1–2

Repository lakshgupta260-dev/sih2k26 · **Stack** FastAPI · PostgreSQL 16 · SQLAlchemy 2 · Alembic · Docker · **Updated** 01 September 2026

Get the backend running, prove it works, and understand what it does. Written for someone who has just cloned the repo. Covers **Phases 1–2**: scaffolding, configuration, database, Docker, authentication, RBAC, users, projects and audit logging.

1. What exists right now

Area	State
Configuration, logging, error envelope	Working
PostgreSQL + SQLAlchemy 2.x + Alembic	Working, 5 tables
Docker (Postgres 16 + Redis 7 + API)	Working
Authentication — JWT access + refresh, bcrypt	Working
RBAC — ADMIN / PROJECT_MANAGER / SITE_SUPERVISOR	Working
Users, projects, memberships	Working
Audit logging	Working, append-only
Tests	**81 passing**
API surface	**17 endpoints**

Not built yet: schedule ingestion, document processing, AI extraction and matching, progress analytics, ML delay prediction, reports, notifications, WhatsApp, Vapi. See `docs/PROGRESS.md`.

2. Prerequisites

```
docker --version
python3 --version
```

- **Have Docker?** Use Path A. Nothing else to install.
- **No Docker?** Use Path B — needs Python 3.11+ and PostgreSQL 14+.

> Anaconda's Python (3.12/3.13) is newer than these dependency pins target. Prefer Docker, or build the venv with an explicit `python3.11`.

3. Path A — Docker (recommended)

Step 1. Confirm the daemon is running (`docker --version` succeeds even when it isn't):

```
docker ps
```

Step 2. Create your environment file:

```
cd backend
cp .env.example .env
```

`.env` is gitignored, so it does not exist after a fresh clone, and compose will not start without it. No edits needed locally.

Step 3. Build and start:

```
docker compose up --build
```

Use `--build` whenever `requirements.txt` has changed, or the container keeps running the old dependency set. First run takes 3–6 minutes.

Step 4. Confirm these lines:

```
db-1     | database system is ready to accept connections
redis-1  | Ready to accept connections tcp
api-1    | INFO  [alembic.runtime.migration] Running upgrade -> e7bd0ac32b44
api-1    | application_starting
api-1    | database_connection_ok          <- the one that matters
api-1    | Uvicorn running on http://0.0.0.0:8000
```

An access-log line every 30 seconds afterwards is the container healthcheck calling `/api/v1/health`. That is a good sign, not noise.

To stop: `Ctrl + C`, then `docker compose down`. Add `-v` only to wipe the database volume.

---

4. Path B — Native

```
brew install python@3.11 postgresql@16
brew services start postgresql@16
createdb sih26122

cd backend
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

Edit `.env` to match your local PostgreSQL. With a Homebrew install your macOS username is usually the superuser and there is no password:

```
POSTGRES_HOST=localhost
POSTGRES_USER=your-mac-username
POSTGRES_PASSWORD=
POSTGRES_DB=sih26122
```

Then:

```
alembic upgrade head
uvicorn app.main:app --reload
```

Point VS Code at the venv or every import shows a red squiggle: `Cmd + Shift + P` → `Python: Select Interpreter` → the entry ending in `backend/.venv/bin/python`.

---

5. Create the first administrator

This step is required, and it is easy to miss.

Self-registration only ever produces a `SITE_SUPERVISOR`, and creating a user with a higher role requires an existing administrator. So the first one has to be created outside the HTTP API:

```
# Docker
docker compose exec api python -m app.cli create-admin --email you@yourdomain.com

# Native
python -m app.cli create-admin --email you@yourdomain.com

# confirm
python -m app.cli list-admins
```

Omit `--password` to be prompted instead of putting it in your shell history. Re-running against an existing address promotes that user and resets their password, so it doubles as account recovery.

> The address must be genuinely valid. Reserved TLDs such as `.local` and `.test` are rejected — by the CLI as well as the API, so the CLI cannot create an account the API would then refuse to authenticate.

---

6. Verify it works

Health

```
curl -i http://localhost:8000/api/v1/health
curl -i http://localhost:8000/api/v1/health/ready
```

Both `200 OK`; the second reports `"database":"up"`.

The authentication walkthrough

Run these in order. Each line proves a specific rule.

```
B=http://localhost:8000/api/v1

# 1. log in as the admin you just created
ADMIN=$(curl -s -X POST $B/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"you@yourdomain.com","password":"YOUR_PASSWORD"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# 2. self-register — note the role that comes back
curl -s -X POST $B/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"supervisor@yourdomain.com","password":"SuperPass123",
       "full_name":"Site Supervisor"}'
#   -> role is SITE_SUPERVISOR, never anything higher

SUP=$(curl -s -X POST $B/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"supervisor@yourdomain.com","password":"SuperPass123"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# 3. a supervisor cannot create a project
curl -s -o /dev/null -w "%{http_code}\n" -X POST $B/projects \
  -H "Authorization: Bearer $SUP" -H 'Content-Type: application/json' \
  -d '{"code":"NOPE","name":"Nope"}'
#   -> 403

# 4. the admin can
PID=$(curl -s -X POST $B/projects -H "Authorization: Bearer $ADMIN" \
  -H 'Content-Type: application/json' \
  -d '{"code":"PROJ-OIL-PL02","name":"Crude Oil Trunk Pipeline Phase II"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['id'])")

# 5. a non-member gets 404 - not 403
curl -s -o /dev/null -w "%{http_code}\n" $B/projects/$PID \
  -H "Authorization: Bearer $SUP"
#   -> 404

# 6. add them as a member
curl -s -X POST $B/projects/$PID/members -H "Authorization: Bearer $ADMIN" \
  -H 'Content-Type: application/json' \
  -d '{"email":"supervisor@yourdomain.com","role":"SITE_SUPERVISOR"}'

# 7. now they can see it, with their own role attached
curl -s $B/projects/$PID -H "Authorization: Bearer $SUP"
#   -> "my_role": "SITE_SUPERVISOR"

# 8. but still cannot edit it
curl -s -o /dev/null -w "%{http_code}\n" -X PATCH $B/projects/$PID \
  -H "Authorization: Bearer $SUP" -H 'Content-Type: application/json' \
  -d '{"name":"Hijacked"}'
#   -> 403
```

Check the audit trail

```
docker compose exec db psql -U postgres -d sih26122 -c \
  "SELECT action, entity_type, details FROM audit_logs ORDER BY created_at;"
```

Every step above should appear.

Using Swagger

Open **http://localhost:8000/docs**.

1. Click **Authorize** (top right).
2. Enter your admin email in the **username** field and your password.
3. Click Authorize, then Close.

Every endpoint is now callable with **Try it out**. The `/auth/token` form-encoded endpoint exists purely to make that button work; application clients should use `/auth/login` with JSON.

Run the tests

```
cd backend
source .venv/bin/activate          # Path B only
pytest -v
```

Expect **81 passed**. Tests create and use a separate `sih26122_test` database so they can never touch your development data, and each test runs inside a transaction that is rolled back afterwards. If PostgreSQL is unreachable the database-backed tests skip with a clear reason rather than failing.

---

7. The API

Health

Method	Path	Access
GET	`/api/v1/health`	public
GET	`/api/v1/health/ready`	public

Auth

Method	Path	Access	Purpose
POST	`/api/v1/auth/register`	public	Self-register (always SITE_SUPERVISOR)
POST	`/api/v1/auth/login`	public	JSON login → token pair
POST	`/api/v1/auth/token`	public	Form login, for Swagger's Authorize
POST	`/api/v1/auth/refresh`	public	Rotate refresh token, get new pair
POST	`/api/v1/auth/logout`	any	Revoke one session, or all
GET	`/api/v1/auth/me`	any	The authenticated user
POST	`/api/v1/auth/change-password`	any	Revokes all other sessions

Users

Method	Path	Access
GET	`/api/v1/users`	ADMIN
POST	`/api/v1/users`	ADMIN (may set role)
GET	`/api/v1/users/{id}`	self or ADMIN
PATCH	`/api/v1/users/{id}`	self or ADMIN (profile only)
PATCH	`/api/v1/users/{id}/role`	ADMIN
PATCH	`/api/v1/users/{id}/status`	ADMIN

Projects

Method	Path	Access
GET	`/api/v1/projects`	any (scoped to your memberships)
POST	`/api/v1/projects`	ADMIN or PROJECT_MANAGER
GET	`/api/v1/projects/{id}`	project member or ADMIN
PATCH	`/api/v1/projects/{id}`	project manager or ADMIN
DELETE	`/api/v1/projects/{id}`	project manager or ADMIN (soft delete)
GET	`/api/v1/projects/{id}/members`	project member or ADMIN

POST	`/api/v1/projects/{id}/members`	project manager or ADMIN
PATCH	`/api/v1/projects/{id}/members/{user_id}`	project manager or ADMIN
DELETE	`/api/v1/projects/{id}/members/{user_id}`	project manager or ADMIN

Every response carries `X-Request-ID`. Send your own to trace a call through the logs.

---

8. How authorization works

Two independent layers.

System role (`users.role`) — one of `ADMIN` , `PROJECT_MANAGER` , `SITE_SUPERVISOR` . Gates platform-wide capability: listing users, creating a project at all.

Project role (`project_memberships.role`) — your role *on one project*. A user who is `SITE_SUPERVISOR` system-wide can be the `PROJECT_MANAGER` of a specific project. A `PROJECT_MANAGER` system-wide has **no** access to a project they are not a member of.

Rules worth internalising:

- Project access resolves from **membership**, not system role. `ADMIN` is the single deliberate bypass and behaves as project manager everywhere.
- A non-member receives **404, not 403**. Confirming a project id exists is itself a leak across the tenancy boundary.
- Creating a project enrols the creator as its manager — otherwise they would immediately lose sight of it.
- A project always keeps at least one project manager; the last one cannot be demoted or removed.
- The last active administrator cannot be demoted, and nobody can demote or deactivate themselves.
- Changing a role or deactivating an account revokes that user's refresh tokens, so the change applies immediately rather than at token expiry.

Tokens

Access tokens are short-lived (default 30 minutes) and carry `role` and `email` claims. Refresh tokens last 7 days by default, are single-purpose, and are recorded server-side by `jti` so they can genuinely be revoked — a stateless JWT alone cannot be logged out.

Refresh is **rotating**: presenting a refresh token revokes it and issues a new one. Replaying an already-revoked token is treated as evidence of theft and revokes every session for that user.

The `type` claim is enforced on decode, so a refresh token cannot be replayed as an access token.

---

9. Project layout

```
backend/app/
├── main.py              application factory, middleware, exception handlers
├── cli.py               admin bootstrap  (python -m app.cli)
├── core/                config, constants, errors, logging, security
├── db/                  declarative base, mixins, engine and session
├── models/              SQLAlchemy models — import every module in __init__.py
├── schemas/             Pydantic v2 request/response contracts
├── repositories/        persistence; owns queries
├── services/            business logic; owns transaction boundaries
├── api/v1/              routers only — no business logic here
├── ai/                  LLM and embedding providers   (Phase 5)
├── ml/                  delay prediction, scikit-learn (Phase 7)
├── document_processing/ PDF / Excel / OCR abstractions (Phase 4)
├── integrations/        Meta WhatsApp, Vapi            (Phase 9-10)
├── notifications/       channel-agnostic dispatch      (Phase 8)
└── utils/
```

The rule that keeps this clean: routers parse and validate, services decide, repositories query. A database query in a route handler belongs in a repository.

Models must be imported in `app/models/__init__.py` . Alembic autogenerate walks `Base.metadata` , and a model that is never imported is invisible to it.

---

10. Troubleshooting

Symptom	Cause	Fix
`Cannot connect to the Docker daemon`	Docker Desktop not running	Open it, wait for the whale to settle
`port is already allocated`	Something else on 5432	Set `POSTGRES_PORT=5433` in `.env`, re-run
`env file .env not found`	Fresh clone	`cp .env.example .env`
Container missing new packages	Image not rebuilt	`docker compose up --build`
`value is not a valid email address … reserved name`	Used a `.local` or `.test` address	Use a real domain, e.g. `example.com`
Cannot create a PROJECT_MANAGER	No admin exists yet	`python -m app.cli create-admin …`
`401` on every request	Missing or expired access token	Log in again; default lifetime 30 min
`403` where you expected `200`	Right authentication, wrong role	Check both system and project role
`404` on a project you know exists	You are not a member	Have a manager add you
`error parsing value for field "CORS_ORIGINS"`	Malformed list in `.env`	`http://a,http://b` — no brackets or quotes
Tests skip with "PostgreSQL not reachable"	No database for the test suite	Start Postgres, or run tests in Docker
Imports underlined red in VS Code	Interpreter not selected	`Cmd+Shift+P` → Python: Select Interpreter
`Unable to create '.git/index.lock'`	Interrupted git command	`rm -f .git/index.lock`

11. Command cheat sheet

```
# Docker
docker compose up --build          # build and start (use after dependency changes)
docker compose up -d               # start detached
docker compose logs -f api         # follow just the API log
docker compose ps                  # container status and health
docker compose down                # stop and remove
docker compose down -v             # ... and wipe the database volume
docker compose exec api bash       # shell inside the API container

# Admin bootstrap   (prefix with `docker compose exec api` on Docker)
python -m app.cli create-admin --email you@yourdomain.com
python -m app.cli list-admins

# Migrations
alembic upgrade head               # apply all
alembic downgrade -1               # roll back one
alembic current                    # applied revision
alembic revision --autogenerate -m "add activities"

# Tests
pytest -v                          # verbose
pytest tests/test_auth.py          # one file
pytest -k "rbac"                   # by name
pytest --cov=app                   # with coverage

# Database
docker compose exec db psql -U postgres -d sih26122
\dt                                # list tables
\d users                           # describe a table
```

12. What comes next

Phase 3 — schedule upload, Excel and CSV parsing with configurable column mapping, the L1–L6 activity hierarchy, and activity dependencies: the "planned" half of the planning-to-execution bridge. Full remaining scope is in `PROGRESS.md`;

design decisions in `../backend/README.md`.